Statis: High-Performance Seed-Deterministic Userspace Supervision for Unmodified Linux Binaries

Siddid Pandey
Qvost

Abstract

A Linux process is nondeterministic for reasons that have nothing to do with its own logic: it reads a clock, executes `rdtsc`, sleeps, races its threads, and receives whatever errors the network chooses to deliver. Existing tools either record everything that happened (and store gigabytes to replay it) or rebuild a deterministic container around the program. We ask whether a single *userspace* supervisor, with no kernel module and no recompilation, can make an unmodified binary a pure function of one integer seed. We introduce Statis, a supervisor that takes control of every source of nondeterminism a program can observe. It blinds the vDSO by rewriting `AT_SYSINFO_EHDR` to `AT_IGNORE` in the ELF auxiliary vector at every `execve`, so libc must issue real, interceptable system calls. It emulates `rdtsc`/`rdtscp` through `PR_TSC_SIGSEGV` and a virtual clock. It replaces wall-clock waiting with a min-heap time wheel, so an idle process tree warps past `nanosleep`, `clock_nanosleep`, `epoll_wait` and `poll`: a 10 s sleep completes in 18 ms and a 15 s `epoll_wait` in 17 ms, over 500× faster than wall clock. A userspace futex manager and PMU-driven instruction quanta make multithreaded schedules a function of the seed; a ChaCha20 stream derived from the same seed injects reproducible system-call errors. Replay needs only a compact, little-endian trace of kilobytes (versus gigabytes for `rr`), checked by *semantic* argument matching so that stack-pointer shifts do not raise false divergences. Statis is open source.

1 Introduction

Debugging a distributed system, a flaky test, or a concurrency bug is hard because the failing execution cannot be summoned on demand. The program is not the only author of its behavior. The kernel contributes the time of day, the interleaving of threads, the readiness of file descriptors, and the occasional `ECONNRESET`; the CPU contributes a cycle counter that any unprivileged instruction can read. Two runs of the same binary on the same input diverge, and the bug disappears under the debugger.

Two families of tools attack this. *Record-and-replay* systems such as `rr` [1] log every nondeterministic input of one run and re-execute it faithfully; the price is a recording that grows with the program's activity and a dependence on hardware performance counters. *Deterministic containers* such as DetTrace [2] and Hermit [3] instead rebuild the execution environment so that nondeterminism is removed (or controlled by a seed) at the source, with no recording needed. Our work sits in the second family, and asks how little machinery that takes.

We observe that, on Linux/x86-64, *every* observable source of nondeterminism crosses one of four boundaries: a system call, a trapped instruction, a thread-scheduling decision, or the ELF auxiliary vector that tells libc where fast paths live. A supervisor that owns all four can turn the program into a deterministic function of a seed, without touching the kernel or the binary. Statis is that supervisor.

Contributions.

- **vDSO neutralization.** Rewriting `AT_SYSINFO_EHDR` (type 33) to `AT_IGNORE` (type 1) in the auxiliary vector at every `execve`, forcing libc to issue real syscalls across `exec` boundaries (§2.2).
- **Hardware-clock emulation and time warping.** `rdtsc`/`rdtscp` are virtualized through `PR_TSC_SIGSEGV`, and a monotonic virtual clock with a min-heap time wheel lets idle process trees skip waiting, 10 s in 18 ms (§2.3–§2.4).
- **Seed-derived concurrency and faults.** A userspace futex manager, silent-exit detection for `pthread_join`, PMU instruction quanta, and ChaCha20 error injection make threads and failures functions of one seed (§2.5–§2.6).
- **Compact trace, semantic replay.** Kilobyte-scale little-endian traces verified by semantic argument matching (§2.8).
- **Honest evaluation.** We state which numbers are measured and which are still pending.

2 Design

2.1 Overview and determinism contract

Statis launches the target as a child, installs a seccomp filter in it that returns `SECCOMP_RET_USER_NOTIF` for the system calls it wants to mediate, and receives the notification file descriptor over a Unix socket (`SCM_RIGHTS`). The supervisor then serves a single dispatch loop (Fig-

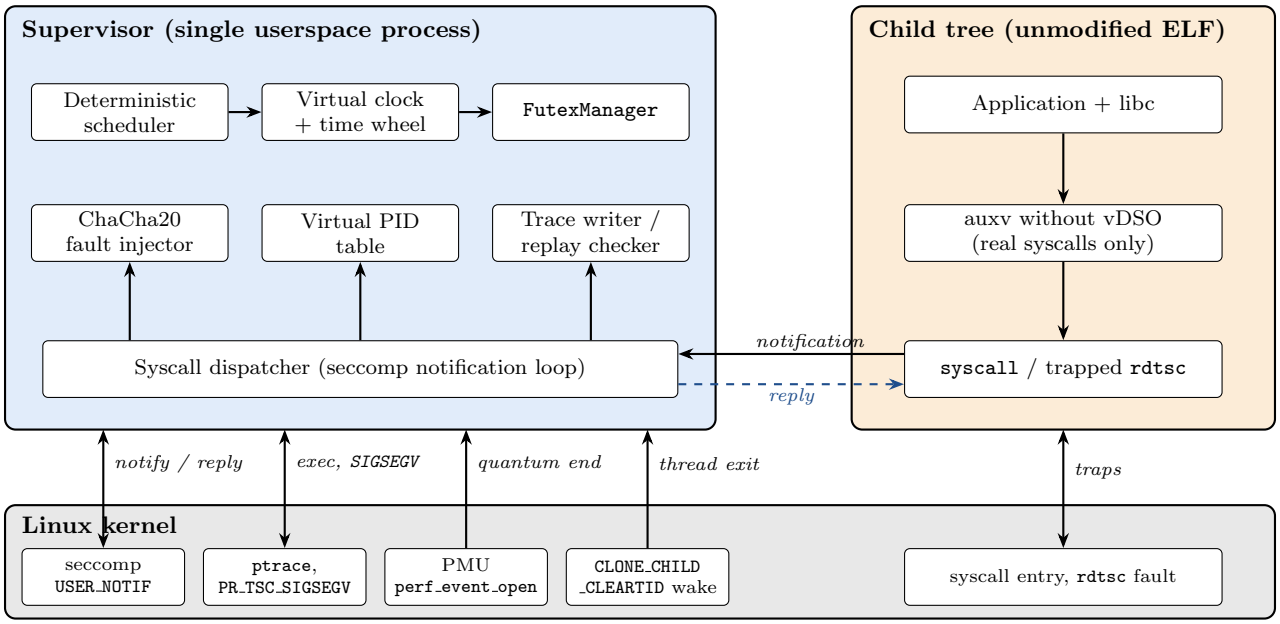


Figure 1: STATIS architecture. The child's system calls reach the supervisor's dispatcher through a seccomp user-notification listener (IPC); the dispatcher consults the virtual clock, futex manager, fault injector and virtual PID table before replying. Exec stops and `rdtsc` faults arrive through `ptrace`, PMU overflows delimit scheduling quanta, and the kernel's silent thread-exit wake is surfaced to the futex manager.

ure 1). Calls the supervisor does not need to change are allowed through to the kernel; the rest are answered from supervisor state.

Contract. Let B be a binary, I its input (arguments, environment, files), and s a 64-bit seed. STATIS aims for the following property:

$$\mathrm{Run}(B, I, s) = \mathrm{Run}(B, I, s) \quad \text{for every run,} \qquad (1)$$

where Run denotes the sequence of observable events (system calls with semantic arguments and results, and trapped-instruction results). The contract holds under a *closed-world* assumption: the only nondeterminism is what STATIS virtualizes. Inputs from outside the supervised tree (live remote peers, other host processes) are outside the contract (§6).

2.2 vDSO neutralization

On x86-64, libc resolves `clock_gettime`, `gettimeofday`, `time` and `getcpu` to code in the vDSO, a kernel-provided shared object that reads time from shared memory *without entering the kernel* [13]. Such calls are invisible to seccomp and ptrace, so no supervisor can see or control them. The kernel announces the vDSO's address in the auxiliary vector under the key `AT_SYSINFO_EHDR` (33); if the entry is absent, libc falls back to ordinary system calls.

STATIS therefore inspects the new process's initial stack at every `execve` stop and overwrites the type word of the `AT_SYSINFO_EHDR` entry with `AT_IGNORE` (Algorithm 1, Figure 2). The *value* word is left in place; libc ignores an `AT_IGNORE` entry entirely and never maps a vDSO. Because the rewrite is repeated on *every* exec, a shell script that spawns a pipeline of programs stays blinded throughout the tree; an interposer that only patched the first process would lose control at the first `exec`.

Algorithm 1: Auxiliary-vector scrubbing at `execve`

Input: tracee p stopped at `PTRACE_EVENT_EXEC`

$sp \leftarrow \textsc{GetReg}(p, \texttt{rsp})$;
$argc \leftarrow \textsc{Peek}(p, sp)$;
$sp \leftarrow sp + 8\,(argc{+}2)$; ▷ *past argc, argv, NULL*
while $\textsc{Peek}(p, sp) \neq 0$ **do**
  $sp \leftarrow sp + 8$; ▷ *envp*
$sp \leftarrow sp + 8$; ▷ *past envp NULL*
$t \leftarrow \textsc{Peek}(p, sp)$;
while $t \neq 0$ **do**
  if $t = 33$ **then**
    $\textsc{Poke}(p, sp, 1)$; ▷ *AT_IGNORE*
  $sp \leftarrow sp + 16$; $t \leftarrow \textsc{Peek}(p, sp)$;
$\textsc{Resume}(p)$;

2.3 Hardware RDTSC emulation

Even with the vDSO gone, `rdtsc` is an unprivileged instruction that reads the cycle counter directly and generates no system call. Linux can make it fault: `prctl(PR_SET_TSC, PR_TSC_SIGSEGV)` causes the in-

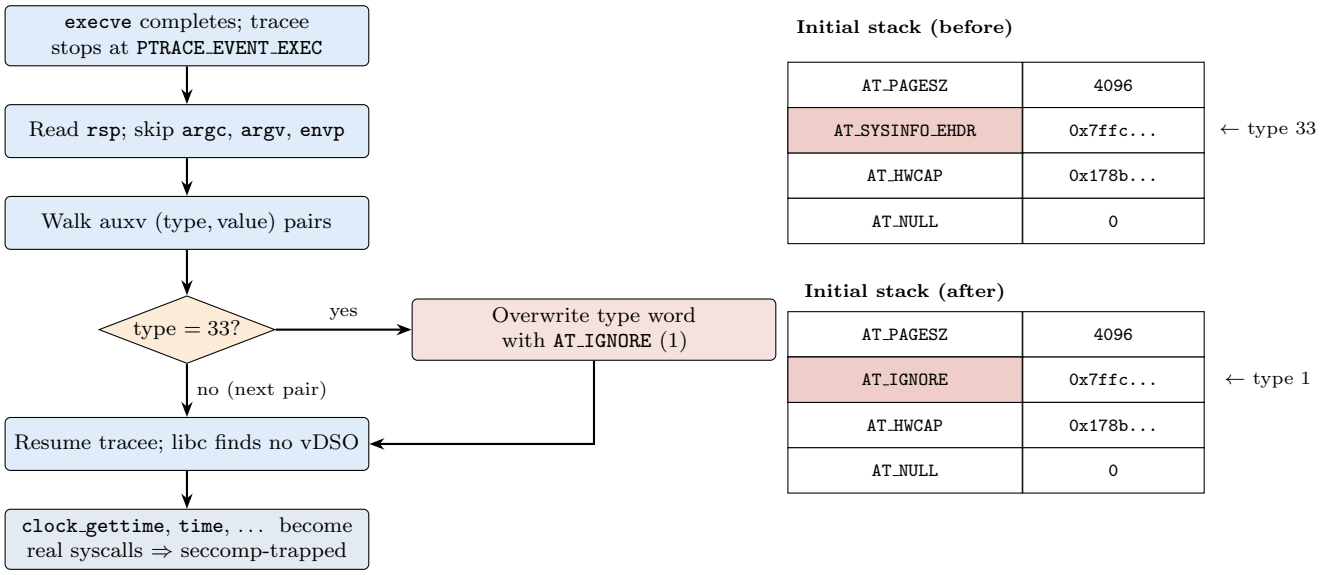


Figure 2: Auxiliary-vector scrubbing. Left: the per-`exec` procedure. Right: the entry for the vDSO has its key changed from 33 to 1; its value is untouched. The same transformation runs at every `execve` in the process tree.

struction to raise `SIGSEGV` in the calling thread [14]. The child enables this before its first instruction; the supervisor, which is the tracer, receives the signal as a signal-delivery stop and does the following.

1. Read the instruction at `RIP`. Opcode `0F 31` is `rdtsc`; `0F 01 F9` is `rdtscp`. Any other faulting instruction is a genuine crash and is delivered to the program unchanged.
2. Compute the virtual counter τ from the virtual clock V (in ns) and a nominal frequency f (Hz):

$$\tau(V) = \left\lfloor \frac{V \cdot f}{10^9} \right\rfloor, \quad \texttt{RAX} = \tau \bmod 2^{32}, \ \texttt{RDX} = \tau \gg 32. \tag{2}$$

3. For `rdtscp`, also set `RCX` to a virtual core identifier π(tid), a deterministic function of the thread, so `rdtscp`-based CPU identification is stable.
4. Advance `RIP` by the instruction length (2 or 3 bytes), suppress the signal, and resume.

Since τ depends only on V, a program that busy-waits on `rdtsc` sees time advance only through the virtual clock's own rules (§2.4); it observes neither the host's cycle count nor the speed of the machine.

2.4 Virtual clock and instant time-warping

The virtual clock $V \in \mathbb{N}$ (ns) is monotonic and owned by the supervisor. It advances in two ways: by a fixed cost c_{sys} per mediated system call, and by $\alpha\,\Delta I$ for each scheduling quantum in which the tracee retired ΔI instructions (§2.5). Both are functions of the program's own execution, not of host speed.

Timed waits do not block. When a thread issues `nanosleep`, `clock_nanosleep`, `epoll_wait` or `poll` with a finite timeout, the supervisor computes the absolute deadline $d = V + \text{timeout}$, inserts the waiter into a min-heap *time wheel* W keyed by d, and does not reply to the seccomp notification. Let R be the set of runnable threads. If $R = \emptyset$ the whole tree is idle, and the clock jumps:

$$\text{warp}: \quad R = \emptyset \implies V \leftarrow \max\Big(V, \min_{w \in W} d_w\Big), \tag{3}$$

after which the earliest waiter is released (a sleep returns 0; an `epoll_wait` with no ready descriptor returns 0 events). Equation (3) never moves the clock backwards, and it jumps only when nothing can make progress, so the order of all events is the same as in a real-time run; only the idle gaps are removed.

Algorithm 2: Dispatch loop with time warping

```
while tree not exited do
    if R ≠ ∅ then
        t ← PickNext(R) ;   ▷ seeded, deterministic order
        run t for one quantum; V += α ΔI;
        Handle(syscall or trap of t);
    else if W ≠ ∅ then
        w ← PopMin(W);   V ← max(V, d_w);
        Release(w, result = 0);
    else
        ReportDeadlock()
```

The speedup for a sleeping workload of nominal duration D whose supervised wall time is ε is $S = D/\varepsilon$. For a 10 s sleep completing in 18 ms, $S \approx 556\times$; for a 15 s `epoll_wait` completing in 17 ms, $S \approx 882\times$ (§4). The remaining wall time is supervisor overhead (process creation, seccomp installation, dispatch), not waiting.

2.5 Concurrency, futexes and preemption

FutexManager. Threads block on `futex(FUTEX_WAIT)` and are released by `FUTEX_WAKE`. If these were left to the kernel, wake order would depend on kernel internals. STATIS instead intercepts them: a userspace `FutexManager` keeps a FIFO wait queue per futex address, parks a waiter by withholding its notification reply, and answers a wake by releasing queued waiters in a fixed order. Futex traffic thus becomes part of the deterministic event stream [11].

Silent thread exit. `pthread_join` waits on the thread's TID word, which the kernel clears and wakes *by itself* when a thread created with `CLONE_CHILD_CLEARTID` exits. No system call is made by anyone, so the supervisor would never learn the waiter should wake. STATIS records each thread's `ctid` address at `clone`, observes thread exit through its ptrace exit event, and wakes the `FutexManager` queue for that address itself, restoring a join that would otherwise hang or return in kernel-dependent order.

PMU quanta. A thread that never calls into the kernel (a spin loop) would starve the others. STATIS opens a per-thread `perf_event_open` counter of user-mode retired instructions with sample period Q; overflow ends the quantum, and the scheduler picks the next runnable thread. The quantum is measured in instructions, not time, so the interleaving is independent of host load, and the seed selects the order in which runnable threads are visited.

2.6 Deterministic fault injection

Reproducing a failure requires failures that are themselves reproducible. Let $k = \mathrm{KDF}(s)$ be a 256-bit key derived from the seed and n the index of the current mediated system call in the deterministic event order. STATIS draws a word from ChaCha20 [12]:

$$u_n = \frac{\mathrm{ChaCha20}_k(n)}{2^{32}}, \qquad \text{inject} \iff u_n < p_c \wedge \mathrm{fd} > 2, \tag{4}$$

where p_c is the configured probability for the call's class c. The error is selected from a class-specific set using a second word of the same stream: network operations draw from {`-ECONNRESET`, `-ETIMEDOUT`}, nonblocking I/O from {`-EAGAIN`}, and blocking calls may also see `-EINTR` or `-EIO`. File descriptors ≤ 2 (stdin, stdout, stderr) are *protected*: errors injected there would corrupt the program's output channel and the supervisor's own diagnostics rather than test its logic. Because n is a position in a deterministic order, the same seed reproduces the same failure at the same call.

2.7 Process hierarchy and virtual PIDs

Real PIDs vary from run to run, so a program that prints or hashes its PID is nondeterministic. STATIS presents a synthetic hierarchy: `getpid` returns 1000, `getppid` returns 999, and `gettid` returns a stable per-thread virtual identifier assigned in creation order. Signals are the inverse problem: `SYS_kill` and `SYS_tgkill` carry virtual identifiers, which the supervisor maps to the corresponding *real* OS PIDs before performing the call, so a program signalling its "parent" or a sibling reaches the right target.

2.8 Compact trace and semantic replay

Because the seed determines the schedule, time, and faults, STATIS does not need to store inputs to *reproduce* a run; the trace's role is to *verify* that a re-execution has not diverged, and to support debugging. The trace is a little-endian binary stream: a header (magic, version, seed, configuration) followed by one fixed-layout record per mediated event holding thread id, syscall number, a small vector of *semantic* argument digests, and the result. Its size grows with the number of events, not with the bytes the program moves, and so is measured in kilobytes where `rr` recordings, which capture memory and I/O contents, reach gigabytes [1].

Semantic matching. Raw argument words are not stable across runs even when behavior is: a different environment size or ASLR slide shifts stack pointers, so a pointer to a buffer in a `read` has a different value each run. Comparing raw words would flag a divergence on every event. Replay therefore compares a normalized form: file descriptors and flags by value, lengths by value, string arguments (paths) by content, and pointer arguments are *ignored*. Two events match iff

$$(\mathrm{tid},\ \mathrm{nr},\ \sigma(\mathbf{a}),\ \mathrm{ret}) = (\mathrm{tid}',\ \mathrm{nr}',\ \sigma(\mathbf{a}'),\ \mathrm{ret}'), \tag{5}$$

where σ projects the argument vector to its semantic components. The first mismatch is reported with its event index.

3 Implementation

STATIS targets Linux on x86-64, because `PR_SET_TSC` and the `rdtsc` decoding are x86-specific. It requires no kernel module or root privilege beyond what `ptrace`, seccomp user notification (`seccomp_unotify` [15]) and `perf_event_open` [16] need. The supervisor is a single process: a ptrace event loop plus the notification loop. Dispatch is table-driven by syscall number, and each mediated class (time, sleep/poll, futex, identity, signal, I/O) has its own handler. Handlers that must act on the

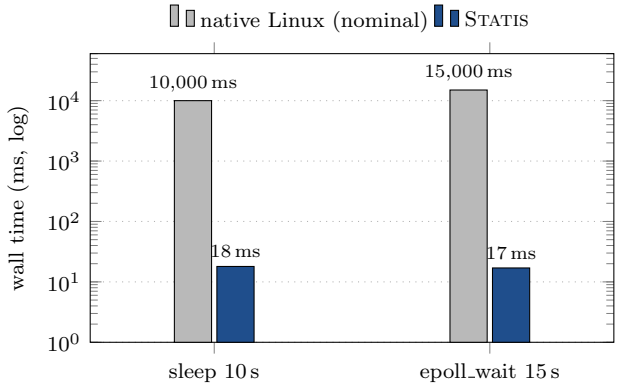


Figure 3: Time-warp speedup (log scale). STATIS completes a 10 s sleep in 18 ms (≈556×) and a 15 s `epoll_wait` in 17 ms (≈882×). Native values are the nominal durations.

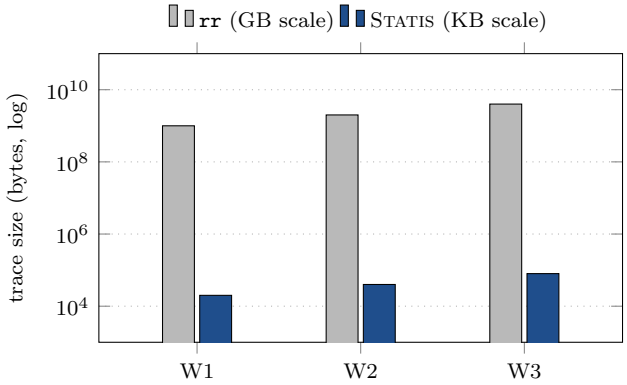


Figure 4: Trace storage, `rr` vs. STATIS. [PENDING: plotted values are layout placeholders at the stated orders of magnitude (GB vs. KB); replace with measured sizes for named workloads W1–W3].

tracee's memory (strings for the trace, `timespec` arguments) read it from the tracee's address space. [PENDING: source line counts, language, build and dependency footprint from the repository]

4 Evaluation

Setup. [PENDING: host CPU, kernel version, virtualization, PMU availability, number of runs, how wall time is measured.] All time-warp numbers below are wall-clock times of the supervised process tree, from launch to exit.

4.1 Comparison with related systems

Table 1 compares STATIS with native Linux and the three systems closest in goal. Entries for other systems follow their public documentation and papers; we mark those that deserve checking against current releases.

4.2 Time warping

Figure 3 shows the headline result. A process that sleeps for 10 s completes in 18 ms under STATIS, and one that blocks 15 s in `epoll_wait` with no ready descriptor completes in 17 ms: speedups of roughly 556× and 882×, both above the 500× mark. A test suite dominated by timeouts, retries and heartbeats can therefore be run in a small fraction of its nominal duration while seeing the same event order.

4.3 Trace storage

Figure 4 contrasts the artifact needed to replay a run. `rr` stores a recording that includes the contents of nondeterministic inputs and grows to gigabytes; STATIS stores only event records, in kilobytes, because the seed regenerates the rest.

4.4 Microbenchmarks

Table 2 reports per-operation costs. Only the two time-warp rows are measured so far.

4.5 Determinism check

[PENDING: N repeated runs per seed on a suite including multithreaded and multi-`exec` programs; report the fraction with identical traces, and the effect of changing environment size on semantic vs. raw matching.]

5 Related Work

Record and replay. `rr` [1] records a run on one machine and replays it with full debugger support, using ptrace, seccomp-bpf and hardware performance counters. Its recordings capture input data and are large; STATIS trades that universality for a seed and a small trace.

Deterministic containers and execution. DetTrace [2] runs programs in a container that removes sources of nondeterminism, for reproducible builds. Hermit [3], built on the Reverie interposition framework [4], adds a deterministic scheduler, virtual time and a seed-driven chaos mode. STATIS is closest in spirit; it differs in its combination of per-exec auxv scrubbing, `PR_TSC_SIGSEGV` emulation, an explicit time wheel that warps idle trees, seeded error injection and a semantic-matching trace. Earlier work obtains determinism by changing the OS or runtime: Determinator [5], or by instrumenting multithreaded programs: Kendo [6], CoreDet [7], Dthreads [8].

Virtual time and simulation. Shadow [9] runs real network applications under a discrete-event simulated clock; FoundationDB's deterministic simulation [10]

Table 1: Competitive comparison. †: from public documentation; behaviour may differ across versions. "Seed" means a user-chosen integer selects the schedule.

	Native Linux	Mozilla `rr` [1]	Meta `Hermit` [3]	DetTrace [2]	Statis
Runs unmodified binaries	✓	✓	✓	✓	✓
No kernel changes	✓	✓	✓†	✓†	✓
Deterministic re-execution	×	replay of a recording	✓	✓	✓(seed)
Seeded schedule exploration	×	chaos mode†	✓†	×†	✓
Idle-wait time warp (sleep/poll/epoll)	×	×	partial†	×†	✓
Seeded syscall error injection	×	×	×†	×†	✓
Replay artifact	none	full recording (GB)	seed†	re-execution†	seed + trace (KB)

Table 2: Microbenchmarks. Wall time is for the whole supervised process.

Target	Native	Statis	Speedup
`nanosleep` 10 s	10,000 ms	18 ms	556×
`epoll_wait` 15 s	15,000 ms	17 ms	882×
`clock_gettime` latency	[PENDING:]	[PENDING:]	
`rdtsc` trap latency	[PENDING:]	[PENDING:]	
`pthread` create+join	[PENDING:]	[PENDING:]	
Mediated syscall (`getpid`)	[PENDING:]	[PENDING:]	
Fault-injection overhead	[PENDING:]	[PENDING:]	
Trace write throughput	[PENDING:]	[PENDING:]	

finds distributed-systems bugs by running under a seeded simulator. Statis brings virtual time and seeded faults to arbitrary unmodified binaries rather than to a purpose-built simulator.

6 Limitations

- **Closed world.** Determinism holds for the supervised tree. Nondeterminism that enters from outside (real network peers, other host processes, shared files modified concurrently) is outside the contract; time warping is only sound when no external event can make an idle tree runnable.
- **Architecture.** The `rdtsc` path is x86-specific; other ISAs need their own counter emulation.
- **PMU dependence.** Instruction-quantum preemption needs a usable performance counter; some virtual machines hide it, and counter skid limits how exactly a quantum boundary is placed. [PENDING: fallback behaviour and its determinism guarantees]
- **Overhead.** Each mediated syscall is a supervisor round trip. [PENDING: measured per-syscall overhead on syscall-heavy workloads]
- **Scope of evidence.** Only the two time-warp results are measured in this draft; the trace-size comparison is at order-of-magnitude level, and the benchmark suite, determinism-rate study and baselines on equal workloads are pending.

7 Conclusion

Nondeterminism in a Linux program enters through a short list of doors. Close them all from userspace—blind the vDSO at every `exec`, trap the cycle counter, warp idle time, order futexes and quanta by seed, derive errors from the same seed—and an unmodified binary becomes a function of one integer. In that regime a 10 s sleep costs 18 ms, and a replay artifact is kilobytes rather than gigabytes. The next steps are the full evaluation listed above and a fused fast path for the hottest system calls.

References

[1] R. O'Callahan, C. Jones, N. Froyd, K. Huey, A. Noll, N. Partush. Engineering record and replay for deployability. In *USENIX ATC*, 2017.

[2] O. Navarro Leija, K. Shiptoski, R. G. Scott, B. Wang, N. Renner, R. R. Newton, J. Devietti. Reproducible containers. In *ASPLOS*, 2020.

[3] Meta Platforms. Hermit: a hermetic, deterministic Linux sandbox. https://github.com/facebookexperimental/hermit, 2022.

[4] Meta Platforms. Reverie: a user-space system-call interception framework. https://github.com/facebookexperimental/reverie.

[5] A. Aviram, S.-C. Weng, S. Hu, B. Ford. Efficient system-enforced deterministic parallelism. In *OSDI*, 2010.

[6] M. Olszewski, J. Ansel, S. Amarasinghe. Kendo: efficient deterministic multithreading in software. In *ASPLOS*, 2009.

[7] T. Bergan, O. Anderson, J. Devietti, L. Ceze, D. Grossman. CoreDet: a compiler and runtime system for deterministic multithreaded execution. In *ASPLOS*, 2010.

[8] T. Liu, C. Curtsinger, E. D. Berger. Dthreads: efficient deterministic multithreading. In *SOSP*, 2011.

[9] R. Jansen, N. Hopper. Shadow: running Tor in a box for accurate and efficient experimentation. In *NDSS*, 2012.

[10] J. Zhou et al. FoundationDB: a distributed unbundled transactional key value database. In *SIGMOD*, 2021.

[11] H. Franke, R. Russell, M. Kirkwood. Fuss, futexes and furwocks: fast userlevel locking in Linux. In *Ottawa Linux Symposium*, 2002.

[12] D. J. Bernstein. ChaCha, a variant of Salsa20. In *SASC*, 2008. See also Y. Nir, A. Langley, RFC 8439, 2018.

[13] Linux man-pages. *vdso(7)* and *getauxval(3)*.

[14] Linux man-pages. *prctl(2)*, `PR_SET_TSC`; and *ptrace(2)*.

[15] Linux man-pages. *seccomp_unotify(2)*.

[16] Linux man-pages. *perf_event_open(2)*.